

IdeaDNA

Guide 2: Evolution Engine — Complete PRD

"Weakness → Mutation → Evolution Tree"

Document:	Evolution Engine — Feature PRD v1.0
Feature:	Core Mode 2 — Idea-to-Evolution Generation
Date:	June 2026
Status:	Final — Ready for Development

1. Feature Overview

Evolution Engine IdeaDNA ka **Unique Selling Proposition (USP)** hai. User koi existing idea daalta hai, system usse analyze karke **4-6 evolved versions** generate karta hai. Har version ek specific enhancement ke saath hota hai: AI-Enhanced, Blockchain-Enhanced, Analytics-Enhanced, Enterprise-Enhanced, etc. Ye versions ek **interactive DNA Evolution Tree** mein visualize hote hain.

Core Philosophy: Pehle weakness identify karo, phir mutation karo, phir evolved versions score karo. Ye 'critic-first' approach hai — most AI tools directly generate, hum pehle analyze karte hain.

2. User Flow & Interaction Design

- **Step 1 — Mode Selection:** User dashboard pe ■ 'Evolve Idea' select karta hai
- **Step 2 — Input:** User apna existing idea describe karta hai (title, description, current features)
- **Step 3 — Critic Analysis:** System weakness, gaps, risks, aur opportunities identify karta hai (visible to user)
- **Step 4 — Evolution Generation:** 4-6 evolved versions generate hote hain, har ek specific mutation tag ke saath
- **Step 5 — Tree Visualization:** React Flow se interactive tree dikhta hai — root = original, branches = evolved versions
- **Step 6 — Selection:** User ek version select karta hai → Deep-dive unlock (Blueprint + UI Mockup)
- **Step 7 — Further Evolution:** Selected version ko aur evolve kiya ja sakta hai (infinite depth)

3. Multi-Agent Architecture (3-Agent SequentialChain)

Agent 1: CriticAgent — Weakness Hunter

Temperature: 0.3 (low = objective, critical). Kaam: Idea ko ruthlessly analyze karna. Kya missing hai? Kya competitors already kar chuke hain? Kya risks hain? Kya next level ho sakta hai?

```
// CriticAgent System Prompt You are a ruthless product critic and venture capitalist. Analyze this idea brutally. Idea: {original_idea} User Profile: {user_profile} Find: 1. WEAKNESSES (what's missing? what could fail?) 2. GAPS (what do competitors already do better?) 3. RISKS (technical, market, legal, scalability) 4. OPPORTUNITIES (what's the next level? what tech could enhance this?) 5. MUTATION VECTORS (specific directions to evolve: AI, Blockchain, Analytics, Social, Enterprise, etc.) Rules: - Be brutally honest, not polite - Every weakness must have a severity: LOW, MEDIUM, HIGH, CRITICAL - Every opportunity must have a potential impact: LOW, MEDIUM, HIGH - Suggest 5-6 specific mutation directions Output: JSON with structured critique and mutation vectors.
```

Agent 2: EvolverAgent — Mutation Specialist

Temperature: 0.6 (medium = structured creativity). Kaam: Critic ki feedback se 4-6 evolved versions banana. Har version ek specific mutation vector follow karega.

```
// EvolverAgent System Prompt You are an Idea Evolution Specialist. Take the original idea + critic's feedback and generate evolved versions. Original Idea: {original_idea} Critic Report: {critic_report} User Profile: {user_profile} Generate 4-6 evolved versions. Each version must: 1. Fix at least one HIGH/CRITICAL weakness 2. Follow one mutation vector from critic's suggestions 3. Have a clear Mutation Tag: AI_ENHANCED, BLOCKCHAIN, ANALYTICS, ENTERPRISE, SOCIAL_IMPACT, IOT, SECURITY, etc. 4. Add specific new features that justify the mutation 5. Keep core concept recognizable 6. Adjust complexity for user's skill level For each version, provide: - versionName (e.g., "AI-Powered Predictive Attendance") - mutationTag (e.g., "AI_PREDICTION") - enhancements (array of new features) - fixes (which weaknesses it addresses) - tradeOffs (what's sacrificed for this enhancement) - estimatedComplexity (EASY/MEDIUM/HARD for this user) Output: JSON array of evolved versions.
```

Agent 3: ScorerAgent — Objective Evaluator

Temperature: 0.2 (very low = objective). Kaam: Har evolved version ko 4 dimensions mein score karna. Same rubric as Idea Generator, par evolution-specific criteria bhi add hote hain.

```
// ScorerAgent System Prompt (Evolution-specific) Score each evolved version on: 1. ORIGINALITY (0-100): How unique is this mutation? Has anyone done this exact combination? 2. FEASIBILITY (0-100): Can the user build this with their constraints? 3. MARKET POTENTIAL (0-100): Does this mutation open new markets? 4. COMPLEXITY MATCH (0-100): Does difficulty align with user's timeline/team? 5. MUTATION STRENGTH (0-100): How well does the mutation enhance the original? (NEW for evolution) Overall = (Originality*0.25 + Feasibility*0.25 + Market*0.20 + Complexity*0.15 + Mutation*0.15) Provide 1-sentence justification for each score. Output: JSON with scores and overall.
```

4. Evolution Tree Visualization (React Flow)

React Flow library se interactive tree banata hai. Har node clickable hai, edges pe mutation tags hain. Node colors score-based hain: Green (80+) = High, Yellow (60-79) = Medium, Red (<60) = Low.

```
// React Flow Node Structure
const nodes = [
  { id: 'root', type: 'custom', position: { x: 400, y: 0 }, data: { label: 'Smart Attendance System', score: 65, type: 'original', color: '#6c757d' } },
  { id: 'ai-version', type: 'custom', position: { x: 100, y: 200 }, data: { label: 'AI-Powered Predictive Attendance', score: 88, type: 'evolved', mutationTag: 'AI_PREDICTION', color: '#28a745' } },
  { id: 'blockchain-version', type: 'custom', position: { x: 400, y: 200 }, data: { label: 'Tamper-Proof Attendance Ledger', score: 82, type: 'evolved', mutationTag: 'BLOCKCHAIN', color: '#28a745' } },
  { id: 'analytics-version', type: 'custom', position: { x: 700, y: 200 }, data: { label: 'Student Performance Analytics', score: 75, type: 'evolved', mutationTag: 'ANALYTICS', color: '#ffc107' } }
];
const edges = [
  { id: 'e1', source: 'root', target: 'ai-version', label: 'AI_ENHANCED', animated: true },
  { id: 'e2', source: 'root', target: 'blockchain-version', label: 'BLOCKCHAIN', animated: true },
  { id: 'e3', source: 'root', target: 'analytics-version', label: 'ANALYTICS', animated: true }
];
```

5. Database Schema — Evolution-Specific

```
model EvolutionTree { id String @id @default(uuid()) projectId String @unique
project Project @relation(fields: [projectId], references: [id]) // Original idea
originalIdea Json // {title, description, features} // Critic report criticReport
Json // {weaknesses, gaps, risks, opportunities, mutationVectors} // Evolved
versions (stored as tree nodes) versions Json // Array of version objects with
mutation tags // Tree structure for React Flow treeData Json // {nodes: [], edges:
[] } // Selected version for deep-dive selectedVersionId String? // Further
evolution (recursive) parentTreeId String? // If this is evolved from another tree
childTrees EvolutionTree[] @relation("TreeEvolution") createdAt DateTime
@default(now()) updatedAt DateTime @updatedAt } // Version-specific blueprint
(lazy-loaded) model EvolutionBlueprint { id String @id @default(uuid()) treeId
String versionId String // Which version this blueprint belongs to // Same
structure as Blueprint model overview String prd Json techStack Json // ... same
fields as Blueprint createdAt DateTime @default(now()) }
```

6. LangChain SequentialChain Implementation

```
// Evolution Engine – 3-Agent SequentialChain
import { SequentialChain, LLMChain }
from "langchain/chains"; import { OpenAI } from "langchain/llms/openai"; import {
PromptTemplate } from "langchain/prompts"; // Agent 1: Critic
const criticChain =
new LLMChain({ llm: new OpenAI({ temperature: 0.3, modelName: "gpt-4" }), prompt:
criticPrompt, outputKey: "critic_report" }); // Agent 2: Evolver
const evolverChain =
new LLMChain({ llm: new OpenAI({ temperature: 0.6, modelName: "gpt-4" }), prompt:
evolverPrompt, outputKey: "evolutions" }); // Agent 3: Scorer
const scorerChain =
new LLMChain({ llm: new OpenAI({ temperature: 0.2, modelName: "gpt-4" }), prompt:
scorerPrompt, outputKey: "scored_evolution" }); // Sequential Pipeline
const evolutionPipeline = new SequentialChain({ chains: [criticChain, evolverChain,
scorerChain], inputVariables: ["original_idea", "user_profile"], outputVariables:
["critic_report", "evolutions", "scored_evolution"] }); // Execute
const result =
await evolutionPipeline.call({ original_idea: JSON.stringify(userIdea),
user_profile: JSON.stringify(userProfile) }); // result.critic_report → Weakness
analysis // result.evolutions → 4-6 evolved versions // result.scored_evolution →
Scored versions
```

7. API Endpoints

Method	Endpoint	Description
POST	/api/evolution/critic	Run CriticAgent on idea
POST	/api/evolution/generate	Full pipeline: Critic → Evolver → Scorer
POST	/api/evolution/evolve	Generate evolved versions from critic report
POST	/api/evolution/score	Score evolved versions
GET	/api/evolution/tree/:id	Get evolution tree data
POST	/api/evolution/select	Select version for deep-dive
POST	/api/evolution/further	Evolve selected version further (recursive)

8. Interview Talking Points

Why Critic-First Approach?

Most AI tools directly generate variations. Hum pehle weakness identify karte hain, phir fix karte hain. Isse evolved versions actually better hote hain, na ki bas different. CriticAgent low temperature (0.3) use karta hai for objective analysis — ye 'yes-man' nahi, 'no-man' hai.

What Makes Evolution 'Real' Evolution?

Har version ek specific weakness fix karta hai. Example: 'Smart Attendance' ka weakness tha 'no backup for face recognition failure' → AI-Enhanced version ne 'attendance prediction' add kiya as backup. Ye random feature nahi, targeted fix hai.

How Does Recursive Evolution Work?

User ek evolved version select karke usse aur evolve kar sakta hai. Har level pe naya critic report, naye mutations. Infinite depth possible — par practical limit 3-4 levels tak rakhte hain. Database mein parent-child relation se track hota hai.

Why React Flow for Visualization?

Tree structure natural hai for evolution. User dekh sakta hai 'original se kya kya bana'. Clickable nodes, animated edges, color-coded scores — ye sab 'wow' factor hai jo judges ko impress karega.

— End of Guide 2: Evolution Engine PRD —